

insurance.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

int propertyValue(GameState game[], int propIndex)
{
    return currentMarketValue(game, propIndex);
}

/* Assumption: repair costs 30% of the property's value. The
   assignment doesn't give an exact figure, so this is used
   consistently everywhere repair cost is needed. */
int repairCost(GameState game[], int propIndex)
{
    return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;
}

int isCovered(InsuranceType policy, DisasterType disaster)
{
    if(policy == NO_INSURANCE)
        return 0;

    if(policy == BASIC_INSURANCE)
        return (disaster == FIRE || disaster == FLOOD);

    return 1; /* Comprehensive and Business Interruption cover everything */
}

int compensationPercent(InsuranceType policy)
{
    if(policy == BASIC_INSURANCE)
        return 80;

    if(policy == COMPREHENSIVE_INSURANCE || policy == BUSINESS_INTERRUPTION)
        return 100;

    return 0;
}

/* Picks a disaster. Every type is equally likely at base, but Heavy
   Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
DisasterType pickDisaster(GameState game[])
{
    int weights[5];
    int total;
    int i;
    int roll;
    DisasterType result;

    weights[FIRE] = DISASTER_WEIGHT;
    weights[FLOOD] = DISASTER_WEIGHT;
    weights[RIOT] = DISASTER_WEIGHT;
    weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;
    weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;

    if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))
        weights[FLOOD] += DISASTER_BOOST;
```

```

    if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))
        weights[RIOT] += DISASTER_BOOST;

    total = 0;

    for(i = 0; i < 5; i++)
        total += weights[i];

    roll = rand() % total;

    result = FIRE;

    for(i = 0; i < 5; i++)
    {
        if(roll < weights[i])
        {
            result = (DisasterType)i;
            break;
        }

        roll -= weights[i];
    }

    return result;
}

void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
{
    int value;
    int premium;
    int premiumPercent;
    char premiumBuf[32];

    if(type == BASIC_INSURANCE)
        premiumPercent = PREMIUM_BASIC_PERCENT;
    else if(type == COMPREHENSIVE_INSURANCE)
        premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
    else if(type == BUSINESS_INTERRUPTION)
        premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
    else
        return;

    value = propertyValue(game, propIndex);
    premium = (value * premiumPercent) / 100;

    /* Rule-LK 34: premiums respond to insurance modifiers. */
    premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;

    if(game[0].players[playerIndex].cash < premium)
        return;

    payMoney(game, playerIndex, premium);

    game[0].board[propIndex].property.insurance = type;
    game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;

    printf("\n%s purchased insurance for %s.\n",
        game[0].players[playerIndex].name,
        game[0].board[propIndex].name);

    formatLKR(premium, premiumBuf);
    printf("Premium : LKR %s.\n", premiumBuf);
}

```

```

/* Any owned property can be insured (Section 2.8) - no longer just
   developed ones. */
int findPropertyToInsure(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance != NO_INSURANCE)
            continue;

        if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)
            return i;
    }

    return -1;
}

/* Finds a policy close to expiring so it can be renewed. */
int findPropertyToRenew(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)
            continue;

        if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)
            return i;
    }

    return -1;
}

void handleInsuranceVisit(GameState game[], int playerIndex)
{
    int propIndex;
    InsuranceType type;

    printf("\n%s landed on an Insurance square.\n",
        game[0].players[playerIndex].name);

    propIndex = findPropertyToRenew(game, playerIndex);

    if(propIndex != -1)
    {
        type = game[0].board[propIndex].property.insurance;

        purchaseInsurance(game, playerIndex, propIndex, type);
    }
}

```

```

        printf("The policy on %s was renewed for another %d rounds.\n",
               game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
        return;
    }

    propIndex = findPropertyToInsure(game, playerIndex);

    if(propIndex == -1)
    {
        printf("No suitable property to insure right now.\n");
        return;
    }

    type = desiredInsurance(game, playerIndex, propIndex);

    purchaseInsurance(game, playerIndex, propIndex, type);
}

/* Rule-LK 11: fixes a disaster-damaged building automatically once
   the owner can afford it. Called at the start of every turn. */
void tryAutoRepair(GameState game[], int playerIndex)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.damaged)
            continue;

        if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)
        {
            int cost;
            char costBuf[32];

            cost = game[0].board[i].property.repairCostOwed;

            formatLKR(cost, costBuf);

            payMoney(game, playerIndex, cost);

            printf("\n%s repaired %s. It can collect rent again.\n",
                   game[0].players[playerIndex].name, game[0].board[i].name);

            printf("Repair Cost : LKR %s.\n", costBuf);

            game[0].board[i].property.damaged = 0;
            game[0].board[i].property.repairCostOwed = 0;
        }
    }
}

/* Writes the name of a disaster into out. */
void disasterName(DisasterType disaster, char out[])
{
    if(disaster == FIRE)
        strcpy(out, "Fire");
}

```

```

    else if(disaster == FLOOD)
        strcpy(out, "Flood");
    else if(disaster == RIOT)
        strcpy(out, "Riot");
    else if(disaster == BUILDING_COLLAPSE)
        strcpy(out, "Building Collapse");
    else
        strcpy(out, "Electrical Failure");
}

/* Pays an insured player's compensation for a disaster and returns
   how much they received. */
int payCompensation(GameState game[], int chosen, int owner, int cost)
{
    int compensation;

    compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;

    /* Political Unrest inflates Business Interruption claims (Rule-LK 34). */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&
        isModifierActive(game, MOD_BI_CLAIMS, -1, -1))
    {
        compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;
    }

    receiveMoney(game, owner, compensation);

    printf("Insurance Claim Approved.\n");
    printf("Compensation Paid : LKR %d\n", compensation);

    /* Business Interruption also covers 5 rounds of lost rental
       income - the repair cost is fully paid above, but the
       property still earns nothing for 5 rounds. */
    if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)
    {
        game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;
        printf("%s will earn no rent for the next %d rounds "
            "(Business Interruption).\n", game[0].board[chosen].name,
            BI_LOST_INCOME_ROUNDS);
    }

    return compensation;
}

/* Rule-LK 10: every 10 rounds, one random developed property may
   be hit by a disaster. */
void triggerDisaster(GameState game[])
{
    int developed[BOARD_SIZE];
    int developedCount;
    int i;
    int chosen;
    DisasterType disaster;
    int owner;
    int cost;
    int compensation;
    char name[32];

    developedCount = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&

```

```

        (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
    {
        developed[developedCount] = i;
        developedCount++;
    }
}

if(developedCount == 0)
    return;

chosen = developed[rand() % developedCount];

disaster = pickDisaster(game);

owner = game[0].board[chosen].property.owner;

printf("\n*** DISASTER ***\n");

disasterName(disaster, name);
printf("%s occurred.\n", name);

printf("Affected Property : %s (owner : %s)\n",
        game[0].board[chosen].name, game[0].players[owner].name);

cost = repairCost(game, chosen);

if(isCovered(game[0].board[chosen].property.insurance, disaster))
{
    compensation = payCompensation(game, chosen, owner, cost);
}
else
{
    compensation = 0;
    game[0].players[owner].sufferedLoss = 1;

    printf("Property was NOT insured against this disaster.\n");
}

game[0].board[chosen].property.repairCostOwed = cost - compensation;

if(game[0].board[chosen].property.repairCostOwed < 0)
    game[0].board[chosen].property.repairCostOwed = 0;

if(game[0].board[chosen].property.repairCostOwed > 0)
{
    game[0].board[chosen].property.damaged = 1;
    printf("%s is damaged and stops earning rent until repaired.\n",
        game[0].board[chosen].name);
}
}

void processInsuranceExpiry(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.insurance == NO_INSURANCE)
            continue;
    }
}

```

```
game[0].board[i].property.insuranceRoundsLeft--;

if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)
{
    printf("\nInsurance policy on %s expires in %d rounds.\n",
           game[0].board[i].name, INSURANCE_EXPIRY_WARNING);
}

if(game[0].board[i].property.insuranceRoundsLeft <= 0)
{
    printf("\nInsurance policy on %s has expired.\n",
           game[0].board[i].name);

    game[0].board[i].property.insurance = NO_INSURANCE;
}
}
```